

Hayden Fuller
S26 Internetworking of things
Lab 6
04/16/26

P1) scripts (comments removed so this doc isn't way too long)

Part 3 – RSA

```
import random
import Number_Package as npkg

gap = random.randint(2**27, 2**29) #

p = npkg.find_prime_smaller_than_k(2**31 - gap)
q = npkg.find_prime_greater_than_k(2**31 + gap)
e = 65537

N = p * q
d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))

print("Alice choose:\np:{p}\nq:{q}\nN:{N}\ne:{e}\nd:{d}".format(p=p, q=q,
N=N, e=e, d=d))

m = random.randint(0, N)
c = npkg.exp_mod(m, e, N)
print("Bob chooses to encrypt plaintext:", m)
print("Bob sends ciphertext          :", c)

m_decrypt = npkg.exp_mod(c, d, N)
print("Alice decrypts Bob's msg and get:", m_decrypt)
```

Part 4 – RSA Protected Socket

Server.py:

```
import argparse
import socket
import json
import re
import Number_Package as npkg
import random
import math

def RSA_encrypt(m, e, N):
    c = npkg.exp_mod(m, e, N)

    return c

def bytearray_xor(key, msg_bytes):
    key = int(key)

    key_bytearray = key.to_bytes(math.ceil(math.log2(key)), byteorder="big")

    key_len, msg_len = len(key_bytearray), len(msg_bytes)

    return bytearray(msg_bytes[i] ^ key_bytearray[i%key_len] for i in
range(msg_len))

parser = argparse.ArgumentParser(description='Lab6.3 Server Program')
parser.add_argument("-p", "--port", type=int, \
                    help="port of the server to connect", required=True)
args = parser.parse_args()

port = args.port

s = socket.socket()
s.bind(('', port))
print("Server is listening to port %d" %(port))

try:
    s.listen(3)

    while True:
        print("="*30)
        print("waiting for connection")
```

```

c, addr = s.accept()
print ('Got connection from', addr)

rdata_bytes = c.recv(1024)
rdata = json.loads(rdata_bytes.decode())

e, N = rdata['e'], rdata['N']

key = random.randint(0, N)
print("server chooses key:", key)

tdata = {'c':RSA_encrypt(key, e, N)}
c.send(json.dumps(tdata).encode())

ciphertext_bytes = c.recv(1024)
plaintext_bytes = bytearray_xor(key, ciphertext_bytes)
print("Client msg:", plaintext_bytes.decode())

plaintext = "Hello, this is the server."
ciphertext_bytes = bytearray_xor(key, plaintext.encode())
c.send(ciphertext_bytes)

c.close()

except ConnectionRefusedError:
    print("Connection refused. You need to run server program first.")
finally:
    print("free socket")
    s.close()

```

```

Client.py:
import argparse
import socket
import json
import re
import math
import Number_Package as npkg
import random

def RSA_keygen():
    gap = random.randint(2**27, 2**29)

    p = npkg.find_prime_smaller_than_k(2**31 - gap)
    q = npkg.find_prime_greater_than_k(2**31 + gap)
    e = 65537

    N = p * q
    d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))

    return e, d, N

def RSA_decrypt(c, d, N):
    m = npkg.exp_mod(c, d, N)

    return m

def bytearray_xor(key, msg_bytes):
    key_bytearray = key.to_bytes(math.ceil(math.log2(key)), byteorder="big")

    key_len, msg_len = len(key_bytearray), len(msg_bytes)

    return bytearray(msg_bytes[i] ^ key_bytearray[i % key_len] for i in
range(msg_len))

parser = argparse.ArgumentParser(description='Lab5.3 Client Program')
parser.add_argument("-s", "--server", type=str, default="127.0.0.1",
                    help="A string of server IP (default: 127.0.0.1)")
parser.add_argument("-p", "--port", type=int,
                    help="port of the server to connect", required=True)
args = parser.parse_args()

ip = args.server

```

```

port = args.port

print("Trying to connect to %s:%d" % (ip, port))

try:
    s = socket.socket()

    s.connect((ip, port))

    e, d, N = RSA_keygen()

    tdata = {'N': N, 'e': e}
    s.send(json.dumps(tdata).encode())

    rdata_bytes = s.recv(1024)
    rdata = json.loads(rdata_bytes.decode())
    c = rdata['c']

    key = RSA_decrypt(c, d, N)
    print("Server's key:", key)

    plaintext = "This is the client. Let's follow your selected key to
communicate!"
    ciphertext_bytes = bytearray_xor(key, plaintext.encode())

    s.send(ciphertext_bytes)

    rciphertext_bytes = s.recv(1024)
    rplaintext_bytes = bytearray_xor(key, rciphertext_bytes)
    print("server msg:", rplaintext_bytes.decode())

except ConnectionRefusedError:
    print("Connection refused. You need to run server program first.")
finally:
    s.close()

```

P2) screenshots

Part 2 – Socket Programming

```
PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.1 Socket\Problem> python3 Server.py -p 1234
Server is listening to port 1234
=====
waiting for connection
Got connection from ('127.0.0.1', 55605)
client: Hello Server
client e: 65537
client N: 4611299948341116473
=====
waiting for connection
Got connection from ('127.0.0.1', 55607)
client: Hello Server
client e: 65537
client N: 4611299948341116473
=====
waiting for connection
|

PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.1 Socket\Problem> python3 Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server: Hello, this is the server
Server: 2438923152802212086
PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.1 Socket\Problem> python3 Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server: Hello, this is the server
Server: 2438923152802212086
PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.1 Socket\Problem> |
```

Part 3 – RSA

```
PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.2 RSA\Problem>
python3 .\RSA-offline.py
Alice choose:
p:1696331863
q:2598635447
N:4408148109067347761
e:65537
d:35985157026614333
Bob chooses to encrypt plaintext: 3288236263801920550
Bob sends ciphertext : 3654181270474547353
Alice decrypts Bob's msg and get: 3288236263801920550
PS C:\Users\Hayden\Desktop\IoT\Lab6\Lab6 - Secure Socket\Lab6.2 RSA\Problem>
python3 .\RSA-offline.py
Alice choose:
p:1637842597
q:2657124709
N:4351952033941429273
e:65537
d:327573422070677585
Bob chooses to encrypt plaintext: 2146210639644006206
Bob sends ciphertext : 768594581363479181
Alice decrypts Bob's msg and get: 2146210639644006206
```

Part 4 – RSA Protected Socket

```
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Server.py -p 1234
Server is listening on port 1234
=====
waiting for connection
Got connection from ('127.0.0.1', 55581)
server chooses key: 1194054135675866868
Client msg: This is the client. Let's follow your selected key to communicate!
=====
waiting for connection
Got connection from ('127.0.0.1', 55582)
server chooses key: 2823683748102631330
Client msg: This is the client. Let's follow your selected key to communicate!
=====
waiting for connection
Got connection from ('127.0.0.1', 63795)
server chooses key: 3548040826490088775
Client msg: Hello fulleh
=====
waiting for connection

PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server's key: 1194054135675866868
server msg: Hello, this is the server.
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server's key: 2823683748102631330
server msg: Hello, this is the server.
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server's key: 3548040826490088775
server msg: Hello, this is the server.
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> |
```

P3) table
Part 3 – RSA

Exercise 1	p	1998847601
	q	2296119677
	N	4589593307980344877
	e	65537
	d	2006025606665969473
	m	10001
	c	3825906542284276017
Exercise 2	N	4400141856896539081
	d	2764011776369096961
	m	123456
	c	2503673653585535070
Exercise 3	p	1940829017
	q	2354138273
	N	4568979870268667641
	e	139241
	d	610100779569588057

P4)

No, RSA is very very slow at this scale, and has a size limit, which would require splitting the file into many many blocks. A file this size should use symmetric encryption that's faster and better suited for large files, where RSA is only needed to share the small symmetric key.

```

P5) script
import argparse
import socket
import json
import re
import Number_Package as npkg
import math
import random

def RSA_encrypt(m, e, N):
    c=npkg.exp_mod(m, e, N)
    return c

def RSA_keygen():
    gap = random.randint(2**27, 2**29)

    p = npkg.find_prime_smaller_than_k(2**31 - gap)
    q = npkg.find_prime_greater_than_k(2**31 + gap)
    e = 65537

    N = p * q
    d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))

    return e, d, N

def RSA_decrypt(c, d, N):
    m = npkg.exp_mod(c, d, N)
    return m

def bytearray_xor(key, msg_bytes):
    key_bytearray = key.to_bytes(math.ceil(math.log2(key)), byteorder="big")

    key_len, msg_len = len(key_bytearray), len(msg_bytes)

    return bytearray(msg_bytes[i] ^ key_bytearray[i % key_len] for i in
range(msg_len))

parser = argparse.ArgumentParser(description='Lab6.4 Mallory Program')
parser.add_argument("-p", "--serverport", type=int,
                    help="port of the server to connect", required=True)
parser.add_argument("-s", "--server", type=str, default="127.0.0.1",
                    help="A string of server IP (default: 127.0.0.1)")
parser.add_argument("-c", "--clientport", type=int,

```

```

        help="port for clients to connect", required=True)
args = parser.parse_args()

server_port = args.serverport
server_ip = args.server
client_port = args.clientport

s_svr = socket.socket()
s_clt = socket.socket()

s_clt.bind('', client_port)
print("Mallory is listening to port %d" % (client_port))

try:
    s_clt.listen(0)

    while True:
        print("="*30)
        print("waiting for victim to connect")

        c, addr = s_clt.accept()
        print('Got connection from victim', addr)

        rdata_bytes = c.recv(1024)
        rdata = json.loads(rdata_bytes.decode())

        e4vic, N4vic = rdata['e'], rdata['N']

        s_svr.connect((server_ip , server_port))

        e4svr, d4svr, N4svr = RSA_keygen()

        tdata = {'N': N4svr, 'e': e4svr}
        s_svr.send(json.dumps(tdata).encode())

        rdata_bytes = s_svr.recv(1024)
        rdata = json.loads(rdata_bytes.decode())
        c4svr = rdata['c']

        key = RSA_decrypt(c4svr, d4svr, N4svr)
        print("Server's key:", key)

        tdata = {'c': RSA_encrypt(key, e4vic, N4vic)}

```

```

c.send(json.dumps(tdata).encode())

ciphertext_from_victim_bytes = c.recv(1024)
plaintext_from_victim_bytes = bytearray_xor(
    key, ciphertext_from_victim_bytes)
print("Victim -> Server:", plaintext_from_victim_bytes.decode())
s_svr.send(ciphertext_from_victim_bytes)

ciphertext_from_server_bytes = s_svr.recv(1024)
plaintext_from_server_bytes = bytearray_xor(
    key, ciphertext_from_server_bytes)
print("Server -> Client:", plaintext_from_server_bytes.decode())
c.send(ciphertext_from_server_bytes)

c.close()
s_svr.close()

except ConnectionRefusedError:
    print("Connection refused. You need to run server program first.")
finally:
    print("free socket")
    s_clt.close()
    s_svr.close()

```

P6) screenshots

```
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Server.py -p 1234
Server is listening to port 1234
=====
waiting for connection
Got connection from ('127.0.0.1', 61065)
server chooses key: 292591580139143572
Client msg: This is the client. Let's follow your selected key to communicate!
=====
waiting for connection

PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.4 MITM Attack\Problem> python3 .\Mallory.py -p 1234 -c 12345
Mallory is listening to port 12345
=====
waiting for victim to connect
Got connection from victim ('127.0.0.1', 61064)
Server's key: 292591580139143572
Victim -> Server: This is the client. Let's follow your selected key to communicate!
Server -> Client: Hello, this is the server.
=====
waiting for victim to connect

PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> python3 .\Client.py -p 12345
Trying to connect to 127.0.0.1:12345
Server's key: 292591580139143572
server msg: Hello, this is the server.
PS C:\Users\Hayden\Desktop\IOT\Lab6\Lab6 - Secure Socket\Lab6.3 RSA Protected Socket\Problem> |
```

P7)

This attack succeeds because neither side verifies who they're talking to. This can be done by not just encrypting with the other's public key and decrypting with your private key, but by also encrypting with your own private key and decrypting with the other's public key. This authenticates so that you can also verify who a message came from. This can also be done by using certificates from a trusted CA and protocols like TLS.